Find Nodes Distance K

AlgoExpert · Hard · Binary Trees

The problem

- Values of all nodes exactly k edges from the node with value `target` (unique values).
- Build parent links, then BFS.

Intuition

- Treat the tree as an undirected graph with parent links, then BFS k steps from the target.

Approach

- 1. Map each node to its parent
- 2. Find the target
- 3. BFS over left, right and parent
- 4. Collect nodes at distance k

Java solution

```java
public class FindNodesDistanceK {
    public static ArrayList<Integer> findNodesDistanceK(BinaryTree tree, int target, int k) {
        Map<BinaryTree, BinaryTree> parent = new HashMap<>();
        BinaryTree start = null;
        ArrayDeque<BinaryTree> st = new ArrayDeque<>();
        st.push(tree);
        while (!st.isEmpty()) {
            BinaryTree n = st.pop();
            if (n.value == target) start = n;
            for (BinaryTree c : new BinaryTree[]{n.left, n.right}) if (c != null) { parent.put(c, n); st.push(c); }
        }
        ArrayList<Integer> out = new ArrayList<>();
        if (start == null) return out;
        Set<BinaryTree> seen = new HashSet<>();
        ArrayDeque<BinaryTree> q = new ArrayDeque<>();
        q.add(start); seen.add(start);
        for (int d = 0; d < k && !q.isEmpty(); d++) {
            for (int s = q.size(); s > 0; s--) {
                BinaryTree n = q.poll();
                for (BinaryTree c : new BinaryTree[]{n.left, n.right, parent.get(n)})
                    if (c != null && seen.add(c)) q.add(c);
            }
        }
        for (BinaryTree n : q) out.add(n.value);
        return out;
    }
```

Complexity

- Time: O(n)
- Space: O(n)

Recap

- Treat the tree as an undirected graph with parent links, then BFS k steps from the target.
- Complexity: O(n)